

Abstract Factory Explanation

This is a copy of a shared ChatGPT conversation

" buttons: First product hierarchy
buttons/Button.java

```
package refactoring_guru.abstract_factory.example.buttons;

/**
 * Abstract Factory assumes that you have several families of products,
 * structured into separate class hierarchies (Button/Checkbox). All products of
 * the same family have the common interface.
 *
 * This is the common interface for buttons family.
 */
public interface Button {
    void paint();
}
```

buttons/MacOSButton.java

```
package refactoring_guru.abstract_factory.example.buttons;

/**
 * All products families have the same varieties (MacOS/Windows).
 *
 * This is a MacOS variant of a button.
 */
public class MacOSButton implements Button {

    @Override
    public void paint() {
        System.out.println("You have created MacOSButton.");
    }
}
```

buttons/WindowsButton.java

```
package refactoring_guru.abstract_factory.example.buttons;

/**
 * All products families have the same varieties (MacOS/Windows).
 *
 * This is another variant of a button.
 */
public class WindowsButton implements Button {

    @Override
    public void paint() {
```

```

        System.out.println("You have created WindowsButton.");
    }
}

checkboxes: Second product hierarchy
checkboxes/Checkbox.java

package refactoring_guru.abstract_factory.example.checkbox;

/**
 * Checkboxes is the second product family. It has the same variants as buttons.
 */
public interface Checkbox {
    void paint();
}

checkboxes/MacOSCheckbox.java

package refactoring_guru.abstract_factory.example.checkbox;

/**
 * All products families have the same varieties (MacOS/Windows).
 *
 * This is a variant of a checkbox.
 */
public class MacOSCheckbox implements Checkbox {

    @Override
    public void paint() {
        System.out.println("You have created MacOSCheckbox.");
    }
}

checkboxes/WindowsCheckbox.java

package refactoring_guru.abstract_factory.example.checkbox;

/**
 * All products families have the same varieties (MacOS/Windows).
 *
 * This is another variant of a checkbox.
 */
public class WindowsCheckbox implements Checkbox {

    @Override
    public void paint() {
        System.out.println("You have created WindowsCheckbox.");
    }
}

factories

```

factories/GUIFactory.java: Abstract factory

```
package refactoring_guru.abstract_factory.example.factories;

import refactoring_guru.abstract_factory.example.buttons.Button;
import refactoring_guru.abstract_factory.example.checkboxes.Checkbox;

/**
 * Abstract factory knows about all (abstract) product types.
 */
public interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}
```

factories/MacOSFactory.java: Concrete factory (macOS)

```
package refactoring_guru.abstract_factory.example.factories;

import refactoring_guru.abstract_factory.example.buttons.Button;
import refactoring_guru.abstract_factory.example.buttons.MacOSButton;
import refactoring_guru.abstract_factory.example.checkboxes.Checkbox;
import refactoring_guru.abstract_factory.example.checkboxes.MacOSCheckbox;

/**
 * Each concrete factory extends basic factory and responsible for creating
 * products of a single variety.
 */
public class MacOSFactory implements GUIFactory {

    @Override
    public Button createButton() {
        return new MacOSButton();
    }

    @Override
    public Checkbox createCheckbox() {
        return new MacOSCheckbox();
    }
}
```

factories/WindowsFactory.java: Concrete factory (Windows)

```
package refactoring_guru.abstract_factory.example.factories;

import refactoring_guru.abstract_factory.example.buttons.Button;
import refactoring_guru.abstract_factory.example.buttons.WindowsButton;
import refactoring_guru.abstract_factory.example.checkboxes.Checkbox;
import refactoring_guru.abstract_factory.example.checkboxes.WindowsCheckbox;

/**
```

```
* Each concrete factory extends basic factory and responsible for creating
* products of a single variety.
*/
```

```
public class WindowsFactory implements GUIFactory {
```

```
    @Override
    public Button createButton() {
        return new WindowsButton();
    }
```

```
    @Override
    public Checkbox createCheckbox() {
        return new WindowsCheckbox();
    }
}
```

```
app
app/Application.java: Client code
```

```
package refactoring_guru.abstract_factory.example.app;
```

```
import refactoring_guru.abstract_factory.example.buttons.Button;
import refactoring_guru.abstract_factory.example.checkboxes.Checkbox;
import refactoring_guru.abstract_factory.example.factories.GUIFactory;
```

```
/**
 * Factory users don't care which concrete factory they use since they work with
 * factories and products through abstract interfaces.
 */
```

```
public class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        button = factory.createButton();
        checkbox = factory.createCheckbox();
    }

    public void paint() {
        button.paint();
        checkbox.paint();
    }
}
```

```
Demo.java: App configuration
```

```
package refactoring_guru.abstract_factory.example;

import refactoring_guru.abstract_factory.example.app.Application;
import refactoring_guru.abstract_factory.example.factories.GUIFactory;
import refactoring_guru.abstract_factory.example.factories.MacOSFactory;
```

```
import refactoring_guru.abstract_factory.example.factories.WindowsFactory;

/**
 * Demo class. Everything comes together here.
 */
public class Demo {

    /**
     * Application picks the factory type and creates it in run time (usually at
     * initialization stage), depending on the configuration or environment
     * variables.
     */
    private static Application configureApplication() {
        Application app;
        GUIFactory factory;
        String osName = System.getProperty("os.name").toLowerCase();
        if (osName.contains("mac")) {
            factory = new MacOSFactory();
        } else {
            factory = new WindowsFactory();
        }
        app = new Application(factory);
        return app;
    }

    public static void main(String[] args) {
        Application app = configureApplication();
        app.paint();
    }
}
```

In abstract factory, how does adding abstract class help? i feel like it makes the structure unnecessarily complex

Show moreShow less

That's a very common reaction. If you only look at the code in this example, the **Abstract Factory** pattern can seem like overengineering.

The key is understanding **what problem it solves**.

Without Abstract Factory

Imagine you write:

```
Java
Button button;
Checkbox checkbox;
```

```
if (os.equals("Windows")) {
    button = new WindowsButton();
    checkbox = new WindowsCheckbox();
} else {
    button = new MacOSButton();
    checkbox = new MacOSCheckbox();
}
```

Simple. Nothing wrong with it.

The problem grows

Now imagine your application has **100 different screens**.

Every screen needs buttons, checkboxes, text fields, menus, dialogs...

Without Abstract Factory, every screen would have code like

```
Java
if (windows) {
    button = new WindowsButton();
    checkbox = new WindowsCheckbox();
    menu = new WindowsMenu();
    textField = new WindowsTextField();
}
else {
    ...
}
```

Now suppose you want to add Linux.

You now have

```
Java
if (windows) { ... }
else if (mac) { ... }
else if (linux) { ... }
```

everywhere.

What the abstract factory does

Instead of asking

Which button should I create?

the application asks

Give me a button.

```
Java
GUIFactory factory = ...

Button b = factory.createButton();
Checkbox c = factory.createCheckbox();
```

The client never knows

- WindowsButton
- MacOSButton
- LinuxButton

Only the factory knows.

The biggest benefit

Notice this constructor:

```
Java
public Application(GUIFactory factory) {
    button = factory.createButton();
    checkbox = factory.createCheckbox();
}
```

The application has **zero knowledge** about Windows or macOS.

Tomorrow you can write

```
Java
class LinuxFactory implements GUIFactory
```

and the application code stays exactly the same.

Only

```
Java
factory = new LinuxFactory();
```

changes.

Why not just use interfaces?

Actually, **the interface is the important part**.

This

```
Java
interface GUIFactory
```

is already abstract.

Making it

```
Java
abstract class GUIFactory
```

wouldn't suddenly make the design better.

Many examples use an interface because there's no shared implementation.

When would an abstract class help?

Suppose every factory does common work.

```
Java
abstract class GUIFactory {

    protected void logCreation(String product) {
        System.out.println(product + " created");
    }

    abstract Button createButton();
    abstract Checkbox createCheckbox();
}
```

Then

```
Java
class WindowsFactory extends GUIFactory {
    ...
}
```

inherits logging.

Interfaces (prior to default methods) couldn't do that.

Nowadays interfaces even support default methods, so the need for an abstract class is smaller.

Why create an entire factory object?

Imagine the application needs **20 UI components**.

Without a factory:

```
Java
new WindowsButton();
new WindowsCheckbox();
new WindowsMenu();
new WindowsDialog();
new WindowsScrollbar();
...
```

The client must know every Windows class.

With a factory:

```
Java
GUIFactory factory = new WindowsFactory();

factory.createButton();
factory.createCheckbox();
```



```
factory.createMenu();
factory.createDialog();
factory.createScrollbar();
```

The client knows only **one concrete class**:

```
Java
WindowsFactory
```

Everything else is abstract.

Another important benefit: consistency

Suppose you accidentally do

```
Java
Button button = new WindowsButton();
Checkbox checkbox = new MacOSCheckbox();
```

Now your UI is mixed.

With Abstract Factory

```
Java
GUIFactory factory = new WindowsFactory();

Button b = factory.createButton();
Checkbox c = factory.createCheckbox();
```

Every product automatically belongs to the same family.

That's one of the main reasons this pattern exists.

Is it overkill?

Yes—often.

If you have

- 2 products
- 2 implementations
- no plans to expand

then Abstract Factory adds extra classes without much benefit.

This pattern shines when you have:

- multiple related product families (Button, Checkbox, Menu, TextField, Dialog, ScrollBar, ...)
- multiple themes/platforms (Windows, macOS, Linux, Dark, Light, Material...)
- the need to swap an entire family at runtime

- a desire to keep the client completely unaware of concrete implementations

So your intuition is correct: for the small example, it feels more complex than necessary. The example is intentionally minimal to demonstrate the pattern's structure. In a real GUI toolkit or large application with many related objects, the abstraction pays off by centralizing object creation, enforcing compatible product families, and making it easier to introduce new families without changing client code.

PracticeQuizFactory creates practicequiz then it is dependency, uml should have filled arrow or open arrow why?

This is a subtle but important UML distinction.

If **PracticeQuizFactory** **creates** **PracticeQuiz** objects, the relationship is usually **not composition** (filled diamond). It is typically a **dependency** (dashed arrow) or simply an implementation detail.

Case 1: Factory creates the object (most common)

```
Java
class PracticeQuizFactory {
    PracticeQuiz createQuiz() {
        return new PracticeQuiz();
    }
}
```

UML:

```
PracticeQuizFactory -.-> PracticeQuiz
                    <<create>>
```

- Dashed arrow = **Dependency**
- Meaning: "PracticeQuizFactory uses PracticeQuiz to create it."
- The factory **does not own** the object after creation.

Why not composition?

Composition (filled diamond) means:

```
Car ◆—— Engine
```

The **Car** permanently **contains** an **Engine**.

- Engine's lifetime depends on the Car.
- If the Car is destroyed, the Engine is destroyed too.
- The Engine is part of the Car's state.

A factory is different.

Example:

```
Java
```

```
PracticeQuiz quiz = factory.createQuiz();
```

After `createQuiz()` returns:

- The quiz belongs to whoever received it.
- The factory doesn't keep it.
- The factory can even be garbage-collected while the quiz continues to exist.

So there is **no "has-a" relationship**.

What if the factory stores the created object?

Suppose:

```
Java
class PracticeQuizFactory {
    private PracticeQuiz quiz;
}
```

Now the factory **has a field** of type `PracticeQuiz`.

The relationship becomes an **association**.

```
PracticeQuizFactory ----- PracticeQuiz
```

If the factory is responsible for the quiz's entire lifetime (rare), then composition could be appropriate.

In the Abstract Factory pattern

Usually you have:

```
<<interface>>
QuizFactory
^
|
PracticeQuizFactory
PracticeQuizFactory -----> PracticeQuiz
<<create>>
```

- **Open triangle (Δ) = Generalization/Inheritance** (`PracticeQuizFactory` implements or extends `QuizFactory`)
- **Dashed arrow = Dependency** (`PracticeQuizFactory` creates/uses `PracticeQuiz`)

Summary

Relationship	UML Symbol	Appropriate here?
<code>PracticeQuizFactory</code> implements <code>QuizFactory</code>	Solid line + open triangle	☒ Yes

Relationship	UML Symbol	Appropriate here?
PracticeQuizFactory creates PracticeQuiz	Dashed arrow (dependency, often labeled <<create>>)	✅ Yes
PracticeQuizFactory contains a PracticeQuiz field	Association (solid line)	Only if it stores one
PracticeQuizFactory owns the lifetime of a PracticeQuiz	Composition (filled diamond)	❌ Usually no

So if your **PracticeQuizFactory** simply calls `new PracticeQuiz()` and returns it, the correct UML relationship to **PracticeQuiz** is a **dependency** (dashed arrow), not a composition. The **open triangle** is only used for inheritance/implementation, not for object creation.

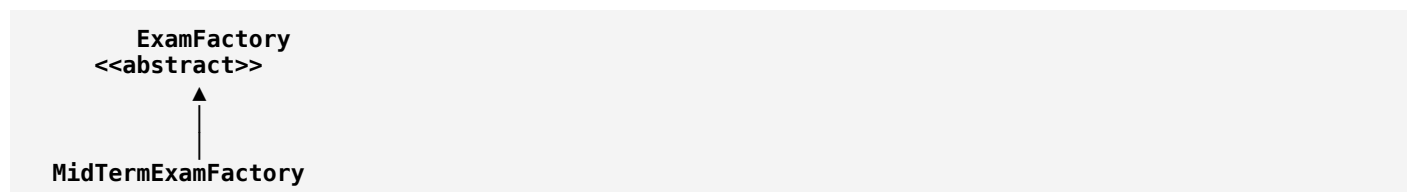
abstract class `ExamFactory` is extended in `MidTermExamFactory` what kind of line and arrow for this

If `MidTermExamFactory` **extends** `ExamFactory`, then the UML relationship is **Generalization (Inheritance)**.

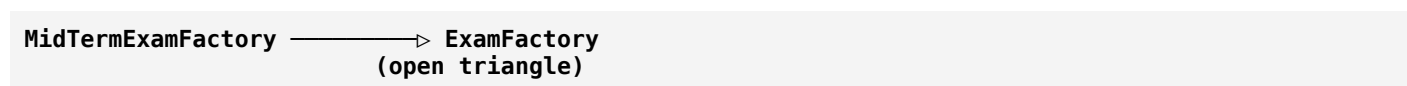
Use:

- **Solid line**
- **Hollow (open) triangle arrow**
- Arrow points **toward the parent** (`ExamFactory`)

Example:



Or in simple ASCII:



UML Notation Summary

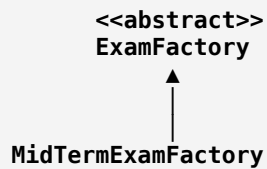
Java Relationship	UML Line	Arrow
extends	Solid line	Hollow (open) triangle
implements	Dashed line	Hollow (open) triangle
Uses/creates (new)	Dashed line	Open arrow
Has-a (field)	Solid line	No arrow (or navigability arrow if desired)
Composition	Solid line	Filled diamond at the owner

So for your code:

```
Java
abstract class ExamFactory { ... }

class MidTermExamFactory extends ExamFactory { ... }
```

The UML should be:



This is **generalization (inheritance)** with a **solid line and a hollow triangle pointing to ExamFactory**.